

SRW3 — Phase 1D-R1: Verifier Soundness Fix & Close Audit

Status: PHASE 1D-R1 EXECUTED — Phase 1D close criteria evaluated against the fixed verifier.

Scope of claim: the verifier domain explicitly modeled here (12-layer independent lineage verifier, Python + K + KEVM). Evidence vocabulary per §15.

1. Executive summary

The scientific question of Phase 1D-R1:

> Can an authorized but dishonest producer create a lineage record that is cryptographically genuine in every field — valid signatures, valid commitments, valid digests — yet does not correspond exactly to the pre-state, effects, and post-state it presents?

Answer: NO — within the modeled verifier domain — after the R1 fixes. Before R1, the answer was YES: the L7 state layer established only $\text{Composed} \subseteq \text{Post}$ (an asymmetric membership loop), and a fully re-signed forged record carrying an extra post-state entry was accepted (reproduced and frozen in Part I, §3). After the Part II fix (canonical finite-map equality `post_map == composed_map`) and the Part VII K-side mirror, the same forgery is rejected by both implementations at the new `invalid-state-composition` verdict (§4, §8, §9).

A second, previously unknown defect was found and fixed during R1: the Phase-1C krypto shim's keccak256 silently produced non-standard digests for all inputs ≥ 136 bytes (finding 1D-R1-KECCAK-MULTIBLOCK, §3.3). This invalidated every K-side CoreHash/Child digest as an Ethereum-keccak value (while leaving within-K consistency intact). Fixed, rebuilt, and byte-validated against the Python reference (§3.3, §13).

All regression surfaces re-verified live with zero decision drift (§14). The permanent adversarial suites are green: L7 suite 10/10 (§4), comparison audit 33 probes with zero deviations (§5), hostile-input harness 21/21 rejected with zero unresolved soundness issues (§7).

2. Baseline and recovery (Part I)

The fifth sandbox reset destroyed the working environment after the Phase-1D close-out push (a2c2dcb). Recovery, in order:

Item	State
Git	origin/phase1c @ a2c2dcb (1D close-out), clean; baseline bundle preserved (main=a857618, kevm-changes=69859b17)
Toolchain	rebuilt from the pinned recipe: K v7.1.337, z3 4.13.3, clang/LLVM-15 15.0.6-4+b1, libsecp256k1 0.5.0 + soname shims, KEVM 1.0.921, plugin @207ae512
Compiled definitions	committed lin-out reused; SRW3-LIN-EVM re-kompiled (interpreter 27.5 MB, size matches the 1D record)
Shim	rebuilt from source: libkrypto-shim.a 18214 B, byte-count identical to the 1D record

Every artifact that Part I could re-verify live was re-verified live (§14). The R1 work that the lost session had not pushed — the independent Python verifier and its attack evidence — was reconstructed from the R1 specification and re-demonstrated (§3).

3. Pre-fix evidence (Part I, frozen)

3.1 The independent Python lineage verifier

python-gen/lin_verify.py — an independent reimplementation sharing no code with the K side: canonical serializations, keccak256 (pycryptodome), secp256k1 sign/recover (coincurve, the libsecp256k1 family the K shim links). Twelve-layer first-fail chain L1..L12 mirroring the K verdict order:

L1 version | L2 tid | L3 parent | L4 appset | L5 inputdigest |
 L6 effectdigest | L7 STATE | L8 policyversion | L9 authority |
 L10 evidence | L11 signature | L12 commitment | VALID

Byte-faithfulness to K was pre-validated early and re-validated after the shim fix: canonical bytes EQUAL (237 B), authority EQUAL, RFC6979 evidence signature EQUAL, and — post-fix — signature and child EQUAL (§13).

3.2 The L7 extra-key attack (reproduced and permanently preserved)

Pre-fix L7 checked (a) `stateD == H(Canon(presented post))` and (b) composition as an **asymmetric membership loop**: every entry of `Composed = Apply(PreState, Effects)` must appear in the presented post-state — extra entries were never rejected. Established relation: `Composed ⊆ Post`, not `Composed == Post`.

The attack (`transcripts/audit/phase1d_r1/l7_attack_prefix.txt`, script `test_lin_attack_L7.py` — both frozen, never to be deleted): `present post' = post u {(app2, slot9): 999}` where slot 9 is in neither pre-state nor effects; recompute `stateD`, evidence, signature, and child with the **valid** key. The pre-fix verdict: **VALID**. Control cases (missing key, changed value, naive stale-stateD) were correctly rejected — demonstrating the precise hole the subset relation leaves.

3.3 Finding 1D-R1-KECCAK-MULTIBLOCK (new, found during Part I)

Byte-level pre-validation between K and Python exposed a divergence in `LinCoreHash`. Isolation (`transcripts/audit/phase1d_r1/FINDING-keccak-multiblock.md`, `srw3lin-hashprobe.k`):

- * the canonical preimage bytes are identical on both sides (237 B, byte-equal);
- * the shim's keccak256 matches standard keccak for 0, 2, 3, 129, 132-byte inputs;
- * for a 136-byte input (exactly the Keccak- κ [1600] rate) the shim returns a **non-standard digest**.

Root cause (`krypto_shim.cpp`): `memset(block, 0, KECCAK_RATE)` before the final `absorb` — `block` aliases the 25-lane state, so the `memset` destroyed the contribution of every absorbed full block. Correct only for inputs shorter than the rate; silently wrong for all inputs ≥ 136 bytes. A faithful Python transcription of the C code reproduces the K output exactly.

Impact: `IntentHash` preimage (100 B) correct; `CoreHash` **(237 B) and Child (306 B) were non-standard keccak values on the K side** for every record ever produced. Within-K consistency (build vs verify) is unaffected — both use the same deterministic H — so all frozen verdicts stand. Cross-layer byte equality fails; the "real Keccak256raw" claim required re-classification (§15). Fix in §4.2.

4. The fixes (Part II)

4.1 L7 exact equality (Python)

The asymmetric loop is removed. L7 now establishes, by **canonical finite-map equality**:

`post_map == composed_map` where `composed_map = flat(Apply(PreState, TrueEffects))`

Canonical equality is symmetric, order-independent, and value-exact — it rejects missing keys, extra keys, and value divergences alike. Asymmetric membership loops are forbidden as equality proofs (mandate discipline; the pre-fix form remains visible in git history at 444672b).

4.2 Shim keccak fix (one line)

The memset is removed; final-block data and the 0x01/0x80 padding are XORed into the live state, as the sponge requires. `libkrypto-shim.a` rebuilt (18022 B); all KRYPTO-dependent definitions re-compiled (`probe-out`, `ck-out`, `lin-out`, `lin-llvm-out`); abstract and 1A/1B KEVM definitions contain no KRYPTO hooks (`grep-verified`) and are untouched. Post-fix: all six hash-probe vectors match standard keccak, including the 136-byte boundary case (7ce759f1ab7f9ce4... on both sides).

5. Full comparison audit (Part IV)

Every comparison site in the verifier, exercised with adversarial inputs (`audit_comparisons.py`, 33 probes). Result: **zero demonstrated deviations** — implemented relations equal intended relations everywhere. Table (abridged to the relation columns; full transcript in the artifact bundle):

Check	Intended relation	Implemented	Verdict
L1 version	<code>version == Int 1 exact</code>	<code>!= 1 → L1-VERSION</code>	OK
L2 tid	<code>tid == Int expected position</code>	<code>!= → L2-TID</code>	OK
L3 parent	32-byte equality vs expected HEAD	<code>!= → L3-PARENT</code>	OK
L4 appset	structural list equality vs ascending-unique apps of TRUE effects (order-significant, not membership)	<code>tuple(appset) != sorted(unique)</code>	OK
L5 inputdigest	digest equality vs claimed pre-state	<code>!= → L5-INPUTDIGEST</code>	OK
L6 effectdigest	digest equality vs TRUE effects (CM-L5)	<code>!= → L6-EFFECTDIGEST</code>	OK
L7(a) stateD	digest equality vs presented post	<code>!= → L7-STATE</code>	OK
L7(b) composition	canonical finite-map equality <code>post_map == composed_map</code>	dict equality after flattening	OK
canonical serialization	order-deterministic, backend-iteration-independent	sorted-key walks, count ₄ prefixes	OK
L8 policyversion	registry membership (necessary, not authorization)	<code>not in → L8-POLICYVERSION</code>	OK
L9 authority	set membership (necessary, never sufficient — CM-L3/L4)	<code>not in → L9-AUTHORITY</code>	OK
L10 evidence	recovered address == authority (cryptographic authorization)	<code>!= → L10-EVIDENCE</code>	OK
L11 signature	recovered address == authority over CoreHash	<code>!= → L11-SIGNATURE</code>	OK
L12 child	digest equality over all 11 non-child fields incl. sig+evidence	<code>!= → L12-COMMITMENT</code>	OK

Audit notes (documented, not deviations): L4 is order-significant by design (the canonical serialization depends on it). The first-fail chain order is load-bearing: an evidence bit-flip with a stale signature fires L10 before L12; a fully re-signed record with a stale child fires L12 — the child covers fields (e.g. `policyV`) that the evidence does not.

6. Presented-effect consistency vs historically true effects (Part V)

Fixing L7 establishes exactly this and nothing more:

> The record's commitments correspond exactly to the **presented** artifact triple (pre-state, effects, post-state), and the presented triple is internally consistent ($\text{post} == \text{Apply}(\text{pre}, \text{effects})$).

It does **not** establish that the presented effects are the **historically true** effects of a real execution: the producer may present a internally-consistent triple describing a transition that never executed, and the verifier — stateless by construction — cannot distinguish it from one that did. Establishing historically true effects requires an authenticated execution witness (the Merkle/MPT authenticated-state-proof interface: NOT YET MECHANIZED, Phase 1E) or chain-level attestation (REQUIRES CLIENT/PROTOCOL SUPPORT). All L7 claims in this report are presented-effect-consistency claims; no stronger claim is made anywhere.

7. Hostile-input mutation harness (Part VI)

21 mutation classes (`audit_mutations_21.py`), authorized-dishonest producer powers applied wherever applicable (digests, signatures and commitments recomputed with the valid key):

Class	Mutation	First-fail layer
m01-m03	version / tid / parent	L1 / L2 / L3
m04-m06	appset +foreign / –drop / order-swap	L4-APPSET
m07	inputD byte-flip	L5-INPUTDIGEST
m08	effectD byte-flip	L6-EFFECTDIGEST
m09	stateD byte-flip	L7-STATE
m10	policyV out of registry	L8-POLICYVERSION
m11	authority → other in-set member (impersonation)	L10-EVIDENCE
m12	authority → out-of-set	L9-AUTHORITY
m13-m14	evidence flip / evidence from another record	L10-EVIDENCE
m15-m16	sig flip / sig from another record	L11-SIGNATURE
m17	child byte-flip	L12-COMMITMENT
m18	ctx: pre-state enriched	L5-INPUTDIGEST
m19	ctx: declared-only effects (CM-L5)	L4-APPSET
m20	ctx: extra post entry, fully re-signed (the L7 attack)	L7-STATE
m21	ctx: registry shrunk	L8-POLICYVERSION

Rejected: 21/21; accepted: 0. The stop rule ("UNRESOLVED VERIFIER SOUNDNESS ISSUE") was not triggered.

8. K correspondence (Part VII)

The same exact equality now exists in K (`k/phase1d/srw3lin-r1.k`, additive): `linCtxP` carries the claimed pre-state; `VerifyLineageP` runs the frozen 12-layer chain and then

`LinCompositionOk` $\Leftrightarrow$ `BytesEq(LinCanonKV(post), LinCanonKV(LinApplyEff(pre, effects)))`

— canonical byte equality, never a membership loop. The legacy `linCtx/VerifyLineage` are untouched (version-compatible: a new constructor, a new outer verdict `invalid-state-composition`).

Suite (r1suite, transcript part7_k_composition_suite.txt), scenario identical to the Python L7 fixtures; **all four records built by LinBuildRec — every digest, evidence, signature and child genuine; only the post correspondence differs:**

pos=valid;neg1=invalid-state-composition;neg2=invalid-state-composition;neg3=invalid-state-composition

- * pos — honest exact state → valid (1 positive)
- * neg1 — extra entry (2,9):=999 (the L7 attack) → rejected
- * neg2 — missing composed entries → rejected
- * neg3 — changed composed value → rejected

9. The authorized-dishonest producer (Parts VIII-IX, core security result)

Part VIII (commit/lineage consistency). The attacker class that recomputes every hash and holds valid signatures is exactly the builder of neg1/L7-B: LinBuildRec (K) and the forge fixture (Python) recompute stateD, evidence, signature, and child over arbitrary presented artifacts. The state-inconsistent record is rejected at the composition check by both implementations — VERIFIER-REJECTED (concrete demonstrations; not a proof of collision-resistance, which remains an assumption on keccak256/secp256k1, §15).

Part IX (the R1 stop criterion). "A holder of a valid private key cannot construct a record with `VerifyLineage = true` and `PostState ≠ PreState ∘ Effects`." After the fix this holds **by construction**: in both implementations the only path to the valid verdict requires the composition predicate to evaluate true (`VerifyLineageP` rule structure; `verify_lineage` control flow). The concrete search for a counterexample — the 21-mutation harness plus the L7 suite — found none (0 accepted). Had any mutation passed, the phase would have stayed open per the mandate; it did not.

10. Chain/domain binding (Part X)

Replay threat, defined. (a) Intra-chain positional replay: a record (or an entire valid chain fragment) re-presented at a different position of the same chain. (b) Cross-domain replay: a record chain from chain/domain A re-presented as a chain on chain/domain B.

Existing binding. Intra-chain positional replay is bound positionally: the verifier consumes the expected tid (position) and expected parent (previous child, rooted at the chain root) and fails at L2/L3 otherwise — DEMONSTRATED (replay scenarios R1-R4: verbatim replay → invalid-tid; tid-forged → invalid-parent; fresh-chain insertion → invalid-parent; honest continuation clean).

Explicit decision (not auto-added). The 12-field record schema is **unchanged**; no chain-id field is added. Rationale: the parent field already carries an arbitrary 32-byte anchor; the correct anchor point is the chain root. Hard-coding `LinRoot = 0x00...20` as every chain's root is a demo convention, not a binding mechanism — under it, cross-domain replay is NOT prevented (a valid chain from domain A verifies as a valid chain on domain B). The decision: the chain root **MUST** be a domain-specific commitment (e.g. `Keccak256(chain-id || genesis-parameters)`), supplied to the verifier as the positional root. This is a verifier-input convention plus a genesis-level protocol obligation — classified **REQUIRES CLIENT/PROTOCOL SUPPORT** for cross-domain replay prevention. No cross-domain claim is made in this phase. Schema unchanged ⇒ no version-compatibility impact; a future chain-id field, if ever added, would require one.

11. Record integrity matrix (Part XI)

Coverage of all 12 fields of Λ_i (✓ = covered; – = excluded by design):

Field	Canon serialization	Signature message (Core)	Intent (evidence)	Child commitment	Independent verify
version	✓	✓	–	✓	L1
parent	✓	✓	–	✓	L3
tid	✓	✓	✓	✓	L2
appset	✓	✓	–	✓	L4
inputD	✓	✓	✓	✓	L5
effectD	✓	✓	✓	✓	L6
stateD	✓	✓	✓	✓	L7
policyV	✓	✓	–	✓	L8
authorit y	✓	✓	–	✓	L9
evidenc e	✓	✓	–	✓	L10
sig	– (sign-then-include)	n/a	–	✓	L11
child	– (self)	n/a	–	n/a	L12

No field is omitted. The intent hash deliberately covers the transition outcome (tid || inputD || effectD || stateD); policyV/authority are covered by the signature and the child. The signature covers the evidence (no evidence laundering). The child covers the signature (no signature substitution). Independent verifiability of every field is exercised by the 12/12 distinct-verdict tamper matrix (Part XII).

12. Replay / tamper matrix re-run (Part XII)

Re-run live on the post-fix stack (transcripts/audit/phase1d_lin_demos.txt, regenerated): the 12-field tamper matrix yields **12 distinct verdicts, one per field**; replay scenarios R1 (verbatim re-presentation → invalid-tid), R2 (tid forged to current position → invalid-parent), R3 (chain record forged into a fresh chain → invalid-parent), R4 (honest continuation after rejected replays → clean full-chain verification). Together with the declared-only substitution (CM-L5, invalid-appset) and the impersonation substitution (CM-L4, invalid-evidence), fifteen substitution classes all fail at their documented verifier layers. All verdict strings are byte-identical to the pre-fix frozen transcript — the shim fix changed digest values (they were non-standard before), not any decision.

13. Three-layer consistency (Part XIII)

* **Python ↔ K (byte level)**, record 1 of lin_true_effects (Q={1:{0:25}}, E={1:{0:25},2:{1:7}}, P={1:{0:25},2:{0:50,1:7}}): inputD, effectD, stateD, authority, evidence (RFC6979), signature (byte-equal and recovery-consistent), child — **8/8 MATCH** (post-fix; pre-fix the CoreHash/Child diverged, which is what exposed the shim defect).

* **K ↔ KEVM**: the KEVM composition (srw3-lin-evm.k) constructs records with the same frozen K modules (LinBuildRec, LinCanonKV) over REAL EVM storage diffs; the lin demos run green on the rebuilt definition (lin_evm_multi n=1, head now standard-keccak 899b6716...; overflow 0 records).

* **Invalid record across layers**: the overflow transition commits no record (gate/obligation layer, KEVM); the tamper matrix and the composition negatives localize the rejection layer (K + Python).

14. Regression freeze (Part XIV)

Live re-verification after recovery and again after the fixes: Python baseline 15/15; abstract demos 13/13 (verdict sequence byte-identical to frozen v5); gen demos 10/10 hs + 10/10 llvm; Python gen mirror 14/14; KEVM gen COMMIT / REJECT+RESTORE; 1C probe 8/8 + ck demos commit/reject/commit; 1D lin demos 6/6; KEVM lin demos green. kprove-heavy stages: the sha-chained frozen v5/v6 records stand (sources unchanged). **Zero decision drift.** Schema was not changed (additive constructor only); no schema-versioned regression was required.

15. Evidence classification

Used exclusively in this report: **PROVED BY K** (structural claims, e.g. chain-continuity base+step, 1D record), **DEMONSTRATED BY K** (concrete krun suites), **DEMONSTRATED BY KEVM** (real-EVM compositions), **VERIFIER-REJECTED** (concrete forgeries rejected by the fixed verifier), **ASSUMED** (keccak256 collision resistance; secp256k1 signature unforgeability without the key; RFC6979 nonce determinism equivalence across libsecp256k1 builds), **REQUIRES CLIENT/PROTOCOL SUPPORT** (cross-domain replay prevention via domain-anchored roots; calldata-level input binding), **NOT YET MECHANIZED** (Merkle/MPT authenticated state proofs — Phase 1E).

Retired overclaims: "commitments = real Keccak256raw" (was false for ≥ 136 -byte preimages — now true post-fix and probe-pinned); any implication that verifier acceptance establishes historically true effects (it establishes presented-effect consistency, §6). The phrase "the verifier is safe" is not used; the demonstrated statement is "the verifier rejects the demonstrated substitution and replay attacks, including fully re-signed ones".

16. Completion criteria (Part XVI)

1. **Exact state binding** — `PostState = Apply(PreState, TrueEffects)` by canonical map equality, both implementations. **MET** (§4, §8).
2. **No asymmetric acceptance** — the subset loop is gone; the audit found zero comparison-site deviations. **MET** (§5).
3. **Authorized-dishonest producer fails** — by construction and by concrete search (21/21 rejected). **MET** (§7, §9).
4. **Cross-layer consistency** — Python $\leftrightarrow$ K byte equality 8/8; K $\leftrightarrow$ KEVM same-module composition. **MET** (§13), within the verifier domain.
5. **All regressions green** — zero decision drift, live. **MET** (§14).
6. **No hidden scope expansion** — scope statements: §6 (presented-effect consistency only), §10 (no cross-domain claim), §15 (assumptions enumerated). **MET**.

Verdict: the Phase 1D close criteria are met within the modeled verifier domain, with the limitations of §6/§10 and the assumptions of §15 explicitly recorded. Merkle/MPT authenticated state proofs remain NOT YET MECHANIZED and are the Phase 1E opening.

17. Artifacts (Part XVII)

Artifact	Path
R1 verifier (post-fix)	python-gen/lin_verify.py
L7 attack (pre-fix, frozen)	python-gen/test_lin_attack_L7.py + transcripts/audit/phase1d_r1/l7_attack_prefix.txt
L7 permanent suite	python-gen/test_lin_verify_L7.py + phase1d_r1/part3_l7_suite_postfix.txt
Comparison audit	python-gen/audit_comparisons.py + phase1d_r1/part4_comparison_audit.txt
21-mutation harness	python-gen/audit_mutations_21.py + phase1d_r1/part6_mutations_21.txt

Artifact	Path
K composition (additive)	k/phase1d/srw3lin-r1.k, srw3lin-r1-demo.k + phase1d_r1/part7_k_composition_suite.txt
Hash probes (permanent)	k/phase1d/srw3lin-hashprobe.k + phase1d_r1/keccak_multiblock_hash_probe.txt
Finding 1D-R1-KECCAK-MULTIBLOCK	phase1d_r1/FINDING-keccak-multiblock.md
Shim (fixed)	k/phase1c/shim/krypto_shim.cpp
Part I/II transcripts	phase1d_r1/part1_baseline_freeze.txt, part2_fix_transcript.txt
Artifact bundle	SRW3-Phase1D-R1-Artifact-Bundle.zip + .sha256 (relative-path MANIFEST)